

Linguaggi Formali e Compilatori

Prof. L. Breveglieri e S. Crespi Reghizzi

Soluzione della Prova scritta 09/09/2005 - Parte I: Teoria

ISTRUZIONI:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche e automi a pila
 3. analisi sintattica
 4. traduzione e semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve superare entrambe le parti (I e II) nello stesso appello oppure in appelli diversi della stessa sessione d'esame.
- Per superare la parte I (teoria) occorre dimostrare sufficiente conoscenza di tutte le quattro sottoparti (1-4).
- Tempo: Parte I (esercitazioni): 30 min - Parte II (teoria): 2.30 ore.
- È permesso consultare libri e appunti personali.

1 Espressioni regolari e automi finiti 20%

1. Data l'espressione regolare di alfabeto $\Sigma = \{a, b, c, d\}$ seguente:

$$R = a(a^*bb \mid ab^*c)^*(d \mid a)$$

- (a) Si dica se l'espressione R sia ambigua oppure no, motivando sinteticamente la risposta.
- (b) Si trovi l'automa deterministico (non necessariamente in forma minima) che riconosce il linguaggio $L(R)$.
- (c) (facoltativo) Si ricavi la forma minima dell'automa deterministico che riconosce il linguaggio $L(R)$.

Soluzione

- (a) Non ambigua
Infatti, se ci fosse ambiguità sarebbe da cercare in

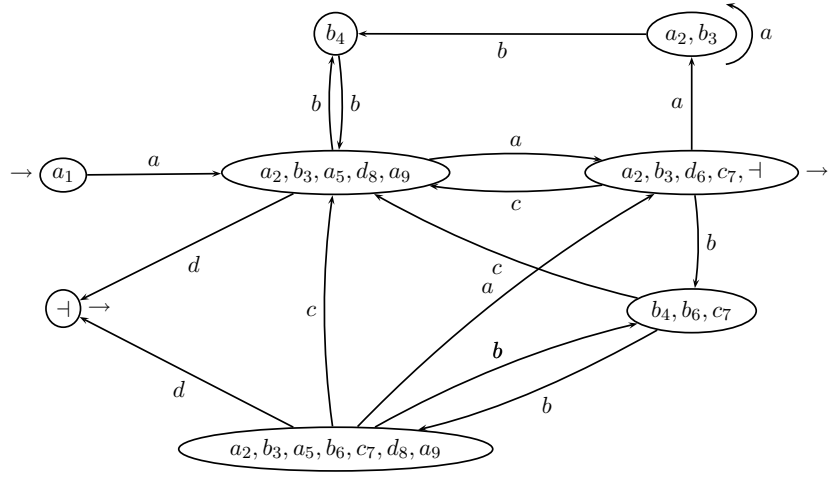
$$\left(\underbrace{a^*bb}_1 \mid \underbrace{ab^*c}_2 \right)^*$$

Si nota che le sotto espr. 1 e 2 danno linguaggi disgiunti, entrambi chiaramente non ambigui. Inoltre la stella di $(1 \mid 2)$ non causa ambiguità perché non è ambiguo il concatenamento 1.2 né il concatenamento 2.1

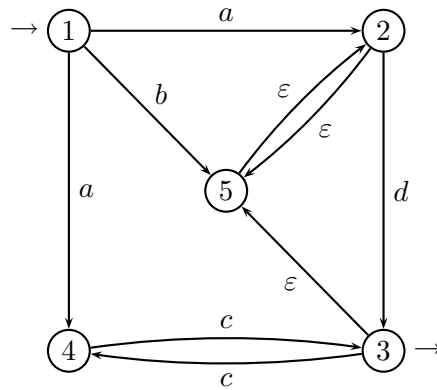
- (b) Con il metodo di Berry e Sethi si costruisce:

$$R' = a_1(a_2^*b_3b_4 \mid a_5b_6^*c_7)^*(d_8 \mid a_9)$$

	Seguiti
a_1	a_2, b_3, a_5, d_8, a_9
a_2	a_2, b_3
b_3	b_4
b_4	a_2, b_3, a_5, d_8, a_9
a_5	b_6, c_7
b_6	b_6, c_7
c_7	a_2, b_3, a_5, d_8, a_9
d_8	$\dashv$
a_9	$\dashv$



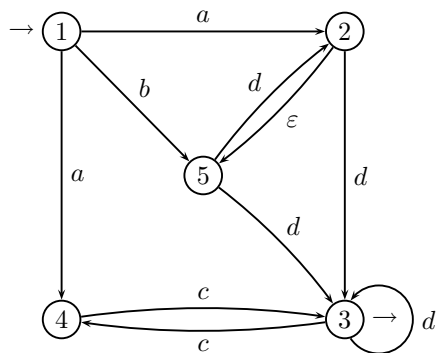
2. È dato l'automa M a stati finiti seguente:



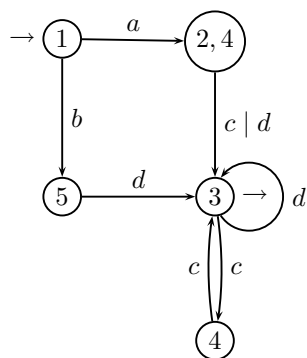
- Si metta l'automa M in forma deterministica (non necessariamente minima).
- Si ricavi un'espressione regolare (non necessariamente non ambigua) equivalente all'automa M .
- (facoltativo) Si dica se l'automa deterministico sia minimo oppure no (non si chiede di trovarne la forma minima), motivando sinteticamente la risposta.

Soluzione

- Eliminazione mosse spontanee



(b) Determinizzazione



(c) È minimo.

2 Grammatiche 20%

1. Si progetti una grammatica in forma non estesa, non ambigua, per il linguaggio delle espressioni aritmetiche polacche prefisse sotto specificato:

- alfabeto $\Sigma = \{+, \times, a\}$
- gli operatori di addizione e moltiplicazione hanno sempre due argomenti
- vi è un vincolo sul numero di moltiplicazioni presenti nell'espressione: non vi può essere più d'una moltiplicazione, a meno che l'espr. contenga soltanto moltiplicazioni, nel qual caso non vi sono limiti al loro numero

Esempi: $++aaa$ $\times \times aa \times aa$ $+\times a + aaa$

Controesempi: $+\times aa \times aa$

- (a) Si scriva la grammatica accertandosi che non sia ambigua.
- (b) Si disegni l'albero sintattico della frase: $+\times a + aaa$

Soluzione

Grammatica:

$S \rightarrow M \mid XPP$

$M \rightarrow +MN \mid +NM \mid +NN \mid \times + NNN \mid \times N + NN$

$N \rightarrow +NN \mid a$

$P \rightarrow \times PP \mid a$

Note: M genera stringhe senza $\times$

N genera stringhe che possono contenere $\times$

P genera stringhe contenenti soltanto l'operatore $\times$

2. Si progetti la grammatica EBNF non ambigua di un frammento di linguaggio Java, così specificato. Sono presenti i concetti seguenti:

- costante intera
- variabile semplice (schematizzata dal carattere terminale i) e array con uno o più indici (racchiusi tra parentesi quadre)
- operatore aritmetico $+$, $-$ (anche unario) e $*$, con la precedenza consueta della moltiplicazione su addizione e sottrazione; non vi sono sottoespressioni parentetizzate
- invocazione di metodo con zero o più argomenti (racchiusi tra parentesi tonde); anche gli identificatori dei metodi e delle classi sono schematizzati con i
- assegnamento
- ogni istruzione è seguita da punto e virgola

Esempio:

$i = i + i[3, i] * i.i(i, i[i]); \quad i = i(i) +;$

- (a) Si scriva la grammatica nella forma BNF estesa.
- (b) Si disegni l'albero sintattico della frase: $i = i + i[3, i] * i.i(i, i[i]);$

3 Grammatiche e analisi sintattica 20%

1. Data la gramm. G seguente:

	Insieme guida
$S \rightarrow ASc$	
$S \rightarrow d$	
$A \rightarrow Ab$	
$A \rightarrow aAb$	
$A \rightarrow \varepsilon$	

- (a) Si calcolino gli insiemi guida $LL(1)$ delle produzioni di G .
- (b) Si spieghino i motivi del non essere $LL(1)$ di G e si scriva una grammatica $LL(1)$ equivalente a quella data. Si calcolino gli insiemi guida delle regole di tale nuova grammatica.
- (c) (facoltativo) Si verifichi se la grammatica G originale sia $LL(k)$ per $k > 1$.

2. Il linguaggio L seguente, di alfabeto $\{a, b, c\}$:

$$L = a^+b^+c \cup \{a^n b^n \mid n \geq 1\}$$

è definito dalla grammatica:

$$S \rightarrow Xc \mid Y$$

$$X \rightarrow Xb \mid Zb$$

$$Z \rightarrow Za \mid a$$

$$Y \rightarrow aYb \mid ab$$

- (a) Si stabilisca se la grammatica sia $LR(0)$, $LALR(1)$ o $LR(1)$.
- (b) Se la gramm. non fosse $LR(1)$, si individui il motivo del conflitto e si scriva una gramm. equivalente di tipo $LR(1)$.

4 Traduzione e semantica 20%

1. Si consideri il linguaggio delle espressioni, formate da variabili $a, b, c, \dots, z$ (lettere minuscole), dall'operatore binario $+$ infisso, e dai due operatori funzionali prefissi $\max 2(e_1, e_2)$ e $\max 3(e_1, e_2, e_3)$ binario e ternario, rispettivamente (i simboli e_i schematizzano variabili o sotto-espressioni); non si usano parentesi per isolare sotto-espressioni.

- (a) Si scriva la schema di traduzione sintattico che traduce le espressioni descritte sopra in espressioni dove l'operatore $+$ è di tipo polacco postfisso e l'operatore $\max$ è solo di tipo binario ma anch'esso di tipo polacco postfisso.

Esempi:

<i>sorgente</i>	<i>pozzo</i>
$\max 3(a, b, c)$	$a \ b \max c \max$
$\max 2(a, b) + \max 3(c, d, e)$	$a \ b \max c \ d \max e \max +$
$a + b + \max 3(c, d, e)$	$a \ b + c \ d \max e \max +$

- (b) Dire poi se si possa mettere lo schema in forma deterministica $LL(1)$, motivando la risposta.

2. Si vuole tradurre in codice macchina un'istruzione d'assegnamento, data in forma polacca prefissa; un es. è la stringa sorgente seguente:

$$:= a_1 + a_2 * a_3 \ a_4$$

il cui significato è: $a_1 \leftarrow a_2 + a_3 * a_4$. La sintassi sorgente è la seguente:

$$S \rightarrow := \ a \ E$$

$$E \rightarrow + \ E \ E$$

$$E \rightarrow * \ E \ E$$

$$E \rightarrow a$$

dove a è il nome di una variabile.

Il codice macchina ha tre sole istruzioni:

istruzione	significato
<i>move</i> $a_1 \ a_2$	$a_2 \leftarrow a_1$
<i>add</i> $a_1 \ a_2 \ a_3$	$a_3 \leftarrow a_1 + a_2$
<i>mult</i> $a_1 \ a_2 \ a_3$	$a_3 \leftarrow a_1 * a_2$

Esempi:

sorgente	pozzo
$:= \ a_1 + a_2 * a_3 \ a_4$	<i>mult</i> $a_3 \ a_4 \ i_1$
	<i>add</i> $a_2 \ i_1 \ a_1$
$:= \ a_5 \ a_6$	<i>move</i> $a_5 \ a_6$

dove i_1 è una cella di memoria il cui nome deve essere creato dalle regole semantiche.

- (a) Si progettino le funzioni semantiche di una gramm. ad attributi per calcolare la traduzione.

Si suggerisce l'uso degli attributi seguenti:

il nome delle variabili: n of a è un attributo lessicale

il nome delle celle di memoria: i of E

il codice prodotto: t of E , t of S

- (b) Si decorino i due alberi con i valori degli attributi e le frecce delle dipendenze funzionali.
- (c) Si verifichi se la valutazione degli attributi possa essere svolta con il metodo L e si scriva, almeno in parte, lo pseudocodice del

valutatore semantico, supponendo che l'albero sintattico sia stato già costruito dal parsificatore.

Sintassi	Funzioni semantiche
$S \rightarrow := a E$	
$E \rightarrow + E E$	
$E \rightarrow * E E$	
$E \rightarrow a$	

